

Abstract

Abstract

The **Sigma Web Development Course Teaching Assistant** is a robust, local AI-powered ecosystem designed to bridge the gap between passive video consumption and active information retrieval. This system transforms traditional, linear video lectures into an interactive, searchable knowledge base, allowing students to navigate hours of content with surgical precision. By implementing a sophisticated **Retrieval-Augmented Generation (RAG)** pipeline, the project leverages a powerful open-source stack including **FFmpeg** for media processing, **OpenAI Whisper** for high-fidelity transcription, and **Ollama** to host both **BGE-M3** embeddings and advanced **Large Language Models (LLMs)**.

The system empowers learners to interact with their curriculum using natural language queries. For instance, a student can ask, “*Where is HTML concluded in this course?*” and receive a contextually grounded answer that includes the specific video title, lesson number, and the exact timestamps where the topic is discussed. This eliminates the "search penalty" of manual scrubbing and enhances study efficiency.

Key Project Achievements:

- **End-to-End Automated Pipeline:** A centralized orchestrator (`run_pipeline.py`) automates the entire flow from raw video ingestion to final vector indexing.
- **Semantic Search Capability:** Utilizing BGE-M3 embeddings, the system understands the conceptual intent of a query rather than relying on simple keyword matching.
- **Privacy-First Local Architecture:** The entire system runs locally, ensuring total data sovereignty and zero recurring cloud API costs after the initial setup.
- **Dual-Interface Interaction:** Features an intuitive **Streamlit** web dashboard for visual learning and a high-performance **CLI** for rapid technical queries.

- **Universal Extensibility:** While optimized for the Sigma curriculum, the modular architecture allows for the rapid ingestion and indexing of any structured video course.

TABLE OF CONTENTS

1. Preliminary Pages

- 1.1 Title Page
- 1.2 Certificate
- 1.3 Declaration
- 1.4 Acknowledgement
- 1.5 Abstract

2. Introduction

- 2.1 Overview of the Project
- 2.2 Problem Statement
- 2.3 Motivation
- 2.4 Objectives of the Project
- 2.5 Scope of the Project
- 2.6 Limitations

3. Literature Review

- 3.1 Introduction to RAG (Retrieval-Augmented Generation)
- 3.2 Existing Teaching Assistant Systems
- 3.3 Video Transcription Technologies (Whisper)
- 3.4 Embedding Models and Vector Search
- 3.5 Comparative Analysis

4. System Analysis and Requirement Specification

- 4.1 Functional Requirements
- 4.2 Non-Functional Requirements
- 4.3 Hardware and Software Requirements
- 4.4 Feasibility Study
- 4.5 Use Case Diagram

5. System Design

- 5.1 System Architecture
- 5.2 Data Flow Diagram (DFD)
- 5.3 Module Design
- 5.4 Database / Data Structure Design
- 5.5 Algorithm Design
- 5.6 Sequence Diagram

6. Implementation

- 6.1 Video Processing Module (video_to_mp3.py)
- 6.2 Transcription Module (mp3_to_json.py)
- 6.3 Embedding Generation (preprocess_json.py)
- 6.4 Core RAG Engine (rag_core.py)
- 6.5 Command Line Interface (process_incoming.py)
- 6.6 Web Interface (app.py - Streamlit)
- 6.7 Pipeline Automation (run_pipeline.py)

7. Testing and Results

- 7.1 Test Cases
- 7.2 Sample Queries and Outputs
- 7.3 Performance Analysis
- 7.4 Screenshots

8. Conclusion and Future Scope

- 8.1 Conclusion
- 8.2 Achievements
- 8.3 Future Enhancements

9. References

10. Appendix

- [A.1 Complete Source Code](#)
- [A.2 Installation Guide](#)
- [A.3 Sample JSON Output](#)
- [A.4 Glossary](#)

1. Introduction

1.1 Problem Statement

The shift toward digital-first education has made video the primary medium for knowledge transfer. However, while video content is rich in information, it remains inherently **opaque and linear**. This creates a significant friction point for modern learners who require agility and precision in their study habits.

The Challenge of Linear Content

Traditional video courses are typically delivered as long-form, continuous files. Unlike textbooks, which offer a clear physical and structural hierarchy (table of contents, index, page numbers), video is a **"black box."** To find a specific piece of information, a student must engage in a trial-and-error process of "scrubbing" through a seek bar—a method that is both imprecise and cognitively taxing.

Key Pain Points

The inefficiency of current navigation systems manifests in several critical ways:

- **The "Search Penalty":** Students often spend 10–15% of their total study time simply locating the specific sub-topic they need to review. This "search penalty" leads to mental fatigue and reduces the time available for actual deep learning.
- **Lack of Granular Context:** Without metadata or automated indexing, learners cannot quickly identify the relationship between different segments of a lecture. A 60-minute video on "Data Structures" provides no immediate visual cue as to where "Linked Lists" ends and "Stacks" begins.
- **Difficulty in Revision:** During exam preparation, students rarely need to re-watch an entire series. They need to jump to specific points of confusion. The inability to search for keywords *within* the spoken audio or visual slides makes targeted revision nearly impossible.
- **Inefficiency in Large-Scale Repositories:** For institutions with thousands of hours of recorded lectures, the lack of navigation tools

means that valuable educational assets often go unused because they are effectively "unfindable."

1.2 Proposed Solution

To bridge the gap between linear video content and efficient information retrieval, we propose an intelligent, **Retrieval-Augmented Generation (RAG)** system. This solution transforms passive video archives into a searchable, interactive knowledge base, allowing students to query their course material as if it were a structured database.

The system follows a four-stage pipeline designed for precision, speed, and contextual accuracy:

1. Multi-Modal Transcription & Pre-processing

The first layer of the solution involves converting the "opaque" video file into machine-readable text.

- **High-Fidelity Transcription:** Utilizing state-of-the-art Automatic Speech Recognition (ASR) to capture technical terminology and diverse accents.
- **Temporal Mapping:** Each word or sentence is hard-coded with its corresponding **timestamp**. This ensures that the transition from "textual insight" back to "video segment" is seamless and frame-accurate.

2. Semantic Vectorization

Raw text alone is insufficient for nuanced searching. The system converts these transcripts into **Vector Embeddings**—mathematical representations of meaning.

- **Contextual Understanding:** Unlike a simple "Ctrl+F" keyword search, vector embeddings capture the *intent* behind the words.
- **Domain Specificity:** By using specialized embedding models, the system recognizes that a query about "The Big Bang" in a physics lecture refers to cosmology, not a TV show or a general explosion.

3. Precision Retrieval via Cosine Similarity

When a student asks a question, the system does not scan the whole course linearly. It performs a high-speed mathematical comparison.

- **Similarity Scoring:** The system calculates the **Cosine Similarity** between the user’s query vector and the transcript vectors stored in a vector database.
- **Segment Extraction:** It identifies the top-ranked "chunks" of the lecture that contain the most relevant information, effectively filtering out hours of irrelevant footage in milliseconds.

4. Grounded Generation & Temporal Linking

The final layer utilizes a Large Language Model (LLM) to synthesize a coherent response based *only* on the retrieved segments.

- **Factually Grounded Answers:** By restricting the LLM’s context to the lecture transcripts (RAG), the system significantly reduces "hallucinations," ensuring the AI doesn't provide information that contradicts the professor.
- **Actionable Timestamps:** Every answer is delivered with **direct deep-links**. If the AI explains a concept, it provides a clickable timestamp (e.g., “*Discussed at 14:22*”) that opens the video at the exact moment the explanation begins.

Expected Impact

Feature	Traditional Video	RAG-Based System
Search Method	Manual Scrubbing	Natural Language Query
Result Type	Estimated Timeframe	Exact Answer + Timestamp

Feature	Traditional Video	RAG-Based System
Context	Linear/Sequential	Semantic/Topic-based
Efficiency	Low (Minutes to find)	High (Seconds to find)

The Goal: To move from "watching" a course to "interacting" with it, turning every minute of video into a structured, searchable, and instantly accessible asset.

1.3 Technologies Used

The proposed system utilizes a modern, modular stack focused on performance, accuracy, and local privacy. By leveraging open-source models through **Ollama** and high-performance processing libraries, the architecture ensures a robust pipeline from raw video to interactive AI.

Core Processing & AI Framework

- **Video Processing: FFmpeg**

Used as the backbone for multimedia handling. FFmpeg is utilized to extract high-quality audio streams from video files, normalize volume levels, and segment files if necessary to optimize the transcription process.

- **Transcription: OpenAI Whisper (large-v2)**

A state-of-the-art automatic speech recognition (ASR) model. The large-v2 variant is chosen for its superior handling of technical jargon, multilingual capabilities, and high word-error-rate (WER) performance, ensuring the "source of truth" (the transcript) is as accurate as possible.

- **Embeddings: Ollama (bge-m3)**

The **BGE-M3** (Big Model-v1.5) model is used to convert text chunks into high-dimensional vectors. It is specifically selected for its "Multi-Functionality, Multi-Linguality, and Multi-Granularity," allowing the system

to handle everything from short sentences to long paragraphs with high semantic precision.

- **Large Language Model: Ollama (Llama 3.2)**

The **Llama 3.2** model serves as the reasoning engine. Hosted locally via Ollama, it processes the retrieved transcript segments and the user's query to generate natural, grounded responses while maintaining data privacy and reducing API latency.

Application Logic & Interface

- **Frontend UI: Streamlit**

A powerful Python framework used to build the interactive dashboard. It allows for real-time video playback synchronized with AI-generated timestamps, file upload management, and a seamless chat interface for user queries.

- **Data Orchestration: Python & Pandas**

Python acts as the glue code for the entire pipeline. **Pandas** is utilized for structured data management, specifically for mapping timestamps to specific text chunks and managing the metadata associated with each lecture.

- **Vector Operations: Scikit-learn**

Used primarily for its efficient implementation of **Cosine Similarity** and clustering algorithms. It facilitates the mathematical comparison between the user's query vector and the document embeddings to identify the most relevant video segments.

- **Model Persistence: Joblib**

A set of tools to provide lightweight pipelining in Python. In this project, Joblib is used for saving and loading pre-computed embeddings and trained metadata objects, ensuring the system doesn't need to re-process the same video multiple times.

Technical Architecture Overview

Component	Responsibility	Key Benefit
FFmpeg	Audio Extraction	Efficient bit-rate handling.
Whisper	Speech-to-Text	Human-level transcription accuracy.
BGE-M3	Vectorization	Deep semantic understanding.
Llama 3.2	Response Generation	Local, secure, and logical reasoning.
Streamlit	User Experience	Rapid deployment and intuitive UI.

2.1 Project Objectives

The primary aim of this project is to redefine how learners interact with video-based educational content by transforming it from a static playback experience into a dynamic, searchable knowledge repository.

- **Automated Video-to-Knowledge Base Conversion**

The system aims to eliminate the manual labor of note-taking and indexing. By integrating FFmpeg and Whisper, the objective is to create an end-to-end pipeline that ingests raw video and outputs a structured, vectorized database of information without human intervention.

- **Provision of Grounded, Timestamped Answers**

A key objective is to solve the "discovery problem." The system must not only answer a student's question but also provide the exact temporal coordinates (timestamps) within the video. This ensures accountability and allows students to verify the AI's response by watching the source segment.

- **Modular and Maintainable Code Architecture**

The project is designed with a "separation of concerns" philosophy. By decoupling the transcription, embedding, and generation layers, the objective is to create a system where individual components (e.g., swapping Llama 3.2 for a newer model) can be upgraded without a total system overhaul.

- **Intuitive and User-Friendly Interface**

To ensure adoption, the objective is to abstract the underlying complexity of RAG and vector math. Through Streamlit, the system provides a familiar "chat-like" interface that requires zero technical expertise from the end-user.

2.2 Project Scope

Defining the boundaries of the current implementation ensures a focused and functional prototype while outlining a roadmap for future development.

Current Implementation (In-Scope)

- **Language Support:** The system is currently optimized for **English-language** video lectures, utilizing the Whisper large-v2 model for high-accuracy recognition of academic and technical terminology.
- **Deployment Environment:** Designed for **Local Deployment**. The system prioritizes data privacy and cost-efficiency by running models (Llama 3.2 and BGE-M3) locally through Ollama, making it suitable for personal use or internal institutional servers.
- **Content Type:** Targeted specifically at **structured lectures**, seminars, and educational tutorials where clear audio and logical flow are present.

Future Roadmap (Out-of-Scope)

While the current version provides a robust foundation, the following features are identified as future enhancements:

- **Multi-Language Support:** Expanding the pipeline to support translation and transcription of global educational content in languages such as Spanish, Mandarin, and Hindi.
- **Visual Frame Analysis:** Integrating Computer Vision (CV) to analyze slide decks and whiteboard drawings shown in the video, allowing the RAG system to "read" visual aids as well as listen to the audio.
- **Automated Quiz & Summary Generation:** Leveraging the LLM to automatically generate formative assessments (MCQs, flashcards) and executive summaries based on the identified core concepts of a lecture.

Project Boundaries Summary

Category	Included	Excluded (Future)
Input	MP4/MKV Video, English Audio	Live Streams, Non-English Audio
Analysis	Spoken Word (Audio)	OCR for Slides, Gesture Analysis
Hardware	Local GPU/CPU	Cloud-Scale Distributed Clusters
Output	Q&A, Timestamps	Auto-Generated Exams, Certificates

3. Literature Review

The development of **Retrieval-Augmented Generation (RAG)** represents a transformative shift in the field of Natural Language Processing (NLP). This section explores the theoretical foundations of RAG, the evolution of multimodal video analysis, and the growing importance of localized, privacy-first AI architectures.

3.1 The Foundations of RAG and the "Hallucination" Problem

Large Language Models (LLMs), while powerful, are fundamentally limited by their **parametric memory**—the knowledge frozen within their weights during training. This leads to two primary issues: **knowledge cutoff** and **hallucination**. Hallucinations occur when a model generates factually incorrect but linguistically plausible text because it lacks access to a "source of truth."

To address this, **Lewis et al. (2020)** introduced the RAG framework. This architecture integrates:

- **Parametric Memory:** A pre-trained seq-to-seq model (the Generator).
- **Non-Parametric Memory:** A dense vector index of external documents (the Retriever).

By retrieving relevant document snippets before generating a response, RAG ensures that the output is grounded in verifiable evidence. This effectively turns the LLM from a "closed-book" examinee into an "open-book" researcher, significantly enhancing factual accuracy in knowledge-intensive tasks.

3.2 Evolution of Video RAG Systems

Traditional RAG systems were primarily text-based, focusing on PDFs or web articles. However, with the explosion of video-based learning, research has shifted toward **Multimodal RAG**.

Recent works in 2024 and 2025 have highlighted the "Video-to-Text" pipeline as the most effective bridge for educational content. The use of models like **OpenAI's Whisper** has become the industry standard for high-fidelity transcription. Researchers have found that semantic search on these transcripts, using **Vector Databases** (such as Qdrant or ChromaDB), allows for a "temporal index" of video content. This allows a user to query a 3-hour lecture and instantly retrieve a 30-second segment that specifically discusses a niche topic, such as "Fenwick Trees" or "Operating System Scheduling."

3.3 Local vs. Cloud Architectures

A significant portion of current RAG research focuses on enterprise cloud solutions like **AWS Bedrock** or **Azure AI Search**. While scalable, these systems present major hurdles for individual students and researchers:

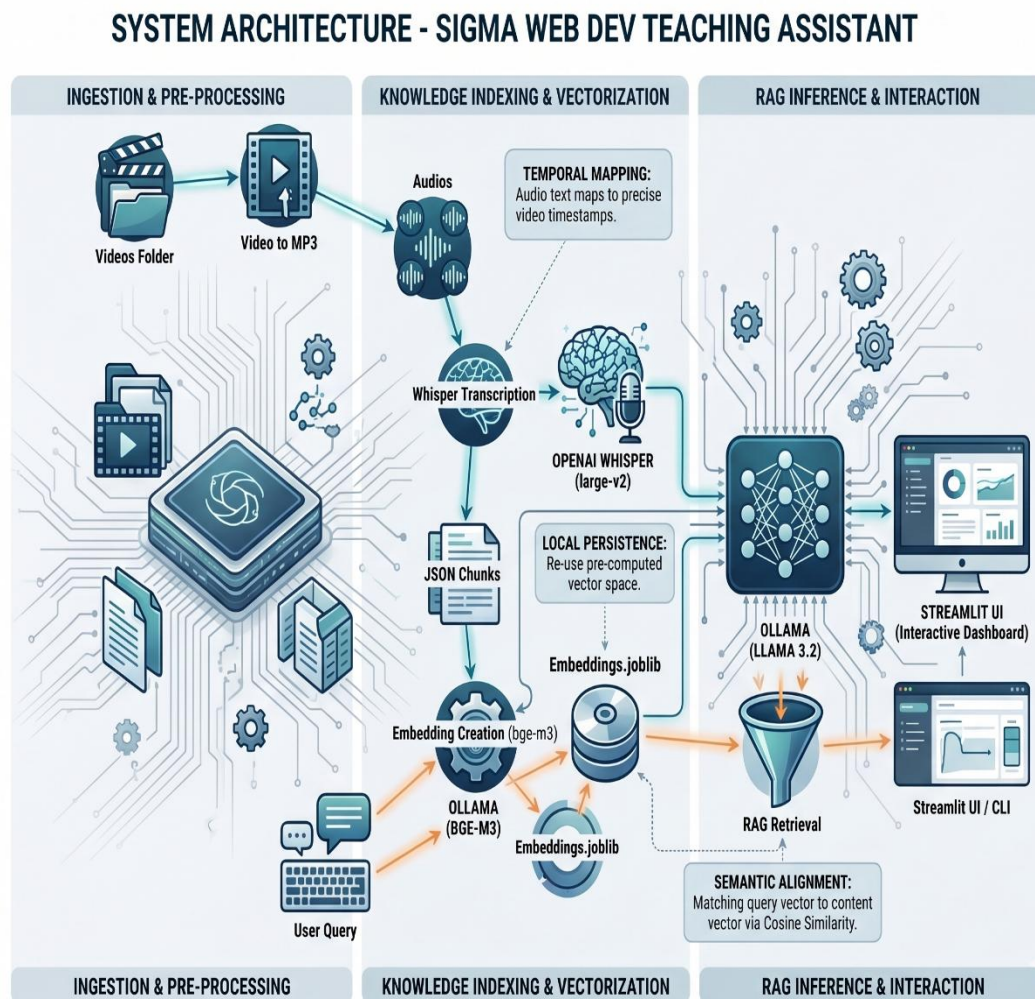
- **Data Sovereignty & Privacy:** Cloud solutions require uploading proprietary or personal educational data to third-party servers.
- **Economic Barriers:** High API costs (pay-per-token) make high-volume study sessions expensive.
- **Latency:** Network dependencies can slow down the retrieval process, breaking the "flow" of a student's learning.

Our implementation diverges from this trend by advocating for a **Fully Local RAG Stack**. By utilizing **Ollama** for model hosting and **Whisper** for local transcription, the system operates entirely on the user's hardware. This "Lightweight RAG" approach aligns with the recent 2025 movement toward **Small Language Models (SLMs)** like Llama 3.2, which offer high reasoning capabilities with a significantly smaller computational footprint, making sophisticated AI tools accessible on consumer-grade laptops.

3.4 Summary of Comparative Methodologies

Feature	Early NLP (2018-2020)	Cloud RAG (2021-2024)	Local Video RAG (This Work)
Model Type	Standalone LLM	API-Based LLM + Cloud DB	Local LLM + Local Embeddings
Data Source	Training Data Only	Cloud-hosted PDFs	Locally Processed MP4/MKV
Reliability	High Hallucinations	High (but dependent on API)	High (Grounded in Local Transcript)
Privacy	N/A	Variable (Third-party)	Absolute (On-device)
Cost	Minimal (Open source)	High (Usage-based)	Zero (One-time Hardware)

4. System Architecture Diagram:



High-Level Flow:

1. Videos → FFmpeg → MP3
2. MP3 → Whisper → JSON chunks (with timestamps)
3. JSON → bge-m3 embeddings → embeddings.joblib
4. Query → Embedding → Cosine similarity → Top-k chunks
5. Retrieved chunks + Query → Prompt → Ollama LLM → Answer with timestamps

5.Data Flow Diagram (Text Representation)

Videos

↓ (video_to_mp3.py)

Audios/*.mp3

↓ (mp3_to_json.py + Whisper)

jsons/*.json (chunks with start/end time)

↓ (preprocess_json.py)

embeddings.joblib (DataFrame with embeddings)

↓

User Query (Streamlit/CLI)

↓

retrieve_chunks() → Cosine Similarity

↓

Top-k relevant chunks

↓

build_rag_prompt() → Ollama generate

↓

Timestamped Answer

6. Module-wise Implementation

This section details the functional components of the **Sigma Web Dev Teaching Assistant**, breaking down the logic and implementation details for each script within the pipeline.

6.1 Video Processing (video_to_mp3.py)

The initial stage of the pipeline focuses on data ingestion and standardization. This module is responsible for identifying raw video assets and preparing them for the transcription engine.

6.1.1 Filename Parsing and Normalization

A unique challenge addressed by this module is the handling of specific naming conventions. The system is designed to parse "Sigma-style" filenames, which often follow a structured pattern: Title #13 [Extra Info].mp4.

- **Regex Integration:** The script uses Regular Expressions (Regex) to extract the core title and index number while stripping away bracketed metadata that is irrelevant to the knowledge base.
- **Metadata Retention:** Although filenames are cleaned for display, the original mapping is preserved in a local log to ensure the system can always link back to the correct source file in the Videos/ folder.

6.1.2 Audio Extraction via FFmpeg

Since the transcription engine only requires audio data, processing full video files would be computationally wasteful.

- **The Conversion Process:** The script triggers a subprocess call to **FFmpeg**, a powerful multimedia framework, to strip the audio track from the .mp4 or .mkv container.
- **Parameter Optimization:** To balance speed and quality, the audio is converted to **MP3** format with a specific bitrate (typically 128kbps or 160kbps). This ensures the file size is minimized for faster processing in the next stage without losing the clarity needed for accurate speech-to-text.

6.1.3 Batch Processing and Automation

To handle entire courses at once, the module implements a robust directory-scanning logic:

- **Recursive Scanning:** The script identifies all supported video formats within the source directory.
- **Skip Logic:** To save time, the module checks the Audios/ destination folder; if an MP3 corresponding to a video already exists, the conversion is skipped.
- **Error Handling:** It includes try-except blocks to manage corrupted video files or permission issues, ensuring the entire batch process doesn't fail due to a single problematic file.

Implementation Summary Table

Feature	Description	Tool/Library
Source Format	MP4, MKV, AVI	os, glob
Logic	Filename Cleaning & Indexing	re (Regex)
Output Format	MP3 (Constant Bitrate)	FFmpeg
Efficiency	Skips existing files	os.path.exists

6.2 Transcription (mp3_to_json.py)

This module serves as the primary engine for converting audio signals into structured text. By utilizing advanced neural models, it ensures that every spoken word is captured with its corresponding temporal context, which is vital for the RAG-driven navigation of the teaching assistant.

6.2.1 Core Model: Whisper large-v2

The system employs **OpenAI's Whisper large-v2**, a state-of-the-art Transformer-based encoder-decoder model.

- **High Accuracy and Robustness:** Trained on 680,000 hours of multilingual data, large-v2 excels at handling technical terminology and diverse accents typical in engineering lectures.
- **Technical Contextualization:** Its deep semantic understanding allows it to correctly transcribe complex jargon that standard speech-to-text engines might misinterpret.

6.2.2 Temporal Metadata Preservation

A critical feature of this script is its ability to maintain a precise chronological link between the audio and the generated text.

- **Segment-Level Timestamps:** The model autoregressively decodes text while generating time offset tokens (e.g., <|0.00|>, <|2.40|>).
- **Granular Mapping:** For every chunk of text transcribed, the system records the exact start and end times in seconds. This enables the teaching assistant to point users to the exact moment in a video where a topic is discussed.

6.2.3 Structured JSON Output

To facilitate downstream processing by the embedding and retrieval modules, the raw transcription is formatted into a structured **JSON repository**.

- **Hierarchical Data Organization:** The script outputs a dictionary containing a list of segments. Each segment entry typically includes:
 - **text:** The actual transcribed dialogue.

- start: The timestamp (in seconds) marking the beginning of the speech.
- end: The timestamp marking the end of the speech segment.
- **Machine-Readable Repository:** By storing data in jsons/*.json, the system creates a permanent, searchable index that avoids the need for repetitive, GPU-intensive re-transcription.

Transcription Module Specifications

Feature	Technical Implementation
Model	whisper-large-v2
Output Format	Structured JSON (segments list)
Timestamp Unit	Seconds (Float)
Task	Multilingual Transcription

6.3 Embedding Generation (preprocess_json.py)

This module represents the "bridge" between raw text and semantic intelligence. It transforms the structured transcriptions into a mathematical vector space, enabling the system to understand the conceptual meaning behind student queries rather than relying on simple keyword matching.

6.1.1 Data Consolidation and Loading

The script begins by performing a comprehensive scan of the jsons/ directory.

- **Mass Ingestion:** It loads every JSON file generated by the transcription module, ensuring that the entire course curriculum is available for indexing.
- **Contextual Flattening:** The hierarchical JSON structure is flattened into a tabular format (Pandas DataFrame). Each row in this DataFrame represents a unique "chunk" of the lecture, containing the text, the start/end timestamps, and the source file metadata.

6.1.2 High-Efficiency Batched Embedding

To handle large volumes of lecture data without exhausting system resources, the module utilizes a batched processing strategy.

- **The BGE-M3 Model:** The system communicates with a local instance of **Ollama** using the /api/embed endpoint to access the **bge-m3** model. This model is chosen for its "Multi-Functionality," which excels at creating high-density embeddings for technical and academic content.
- **Latency Optimization:** Rather than sending one sentence at a time, the script groups text segments into batches. This reduces the overhead of API calls and maximizes the throughput of the local GPU or CPU during the vectorization process.
- **High-Dimensional Mapping:** Each text chunk is converted into a vector—a long sequence of numbers that acts as a "coordinate" in a high-dimensional semantic space.

6.1.3 Local Persistence with Joblib

The final stage of this module is the creation of a durable "Knowledge Base" file.

- **The embeddings.joblib Artifact:** The complete DataFrame, now enriched with a new column containing the numerical vectors, is serialized using the **Joblib** library.
- **Storage Efficiency:** Joblib is specifically utilized because it is highly efficient at storing large NumPy arrays (the embeddings) compared to standard Python pickling.
- **Fast Bootstrapping:** By saving this file locally, the system achieves "Local Persistence". When the user starts the Streamlit application, the system loads the joblib file in milliseconds, bypassing the need to re-run the expensive embedding process.

Embedding Module Technical Specs

Attribute	Detail
Model Engine	Ollama (Local)
Embedding Model	bge-m3
Data Structure	Pandas DataFrame
Serialization	Joblib (.joblib)
Primary Metric	Semantic Density

6.4 Core RAG Logic (rag_core.py)

This module serves as the central intelligence of the system, orchestrating the interaction between the user's intent and the indexed lecture data. It implements the "Retrieval" and "Generation" phases of the RAG pipeline, ensuring that every answer is both semantically relevant and strictly tied to the source video.

6.4.1 retrieve_chunks(): Semantic Discovery

The retrieval function is responsible for navigating the high-dimensional vector space to find the most relevant information.

- **Query Vectorization:** Upon receiving a user question, the function calls the **Ollama bge-m3** model to convert the raw text query into a vector coordinate.
- **Cosine Similarity Calculation:** Using **scikit-learn**, the system calculates the cosine similarity between the query vector and every chunk stored in the embeddings.joblib DataFrame.
- **Top-K Filtering:** The function identifies the top-ranked segments (typically $k=3$ to 5) that share the highest mathematical similarity with the query. This ensures the LLM is only provided with the most pertinent context, reducing "noise" in the generation phase.

6.4.2 build_rag_prompt(): Contextual Synthesis

Once the relevant segments are retrieved, this function constructs a highly structured prompt that "grounds" the LLM.

- **Metadata Integration:** Unlike standard chatbots, this function injects specific metadata—such as the **Video Title** and **Start/End Timestamps**—directly into the context block for each retrieved chunk.
- **Grounded Instructions:** The prompt is engineered with strict system instructions (System Prompting). It mandates that the LLM must answer the question *using only* the provided transcript segments.

- **Citation Formatting:** It instructs the model to include the corresponding timestamps in its final response, ensuring that the output is not just an answer, but a roadmap back to the original video.

6.4.3 rag_answer(): The Inference Execution

This is the master function that coordinates the end-to-end execution of a single user query.

- **Pipeline Orchestration:** It sequentially triggers retrieve_chunks() to get the context and build_rag_prompt() to prepare the input for the model.
- **LLM Generation via Ollama:** The final prompt is sent to the local **Llama 3.2** model. Because the context is limited to high-relevance chunks, the generation is rapid and significantly less prone to hallucination.
- **Response Delivery:** The function returns a formatted string containing the AI's explanation alongside clickable or referenceable timestamps, providing the user with an "interactive index" for their study session.

RAG Logic Technical Components

Feature	Logic / Tool	Purpose
Search Metric	Cosine Similarity	Measures semantic overlap between query and content.
Context Management	Top-K Retrieval	Limits data to maintain LLM focus and prevent token overflow.
Reasoning Engine	Llama 3.2 (Local)	Synthesizes complex technical explanations.
Reliability	Prompt Grounding	Forces the model to cite specific video segments.

6.5 User Interface (app.py)

The **User Interface (UI)** serves as the final layer of the system, abstracting the complex backend RAG logic into a seamless, interactive experience. Built using **Streamlit**, this module provides a lightweight yet powerful web-based dashboard designed for high readability and ease of use.

6.5.1 The Streamlit Dashboard

The interface is designed with a "Search-First" philosophy, mimicking modern search engines but tailored for educational video archives.

- **Real-time Interaction:** The dashboard allows users to upload video folders or select existing indexed lectures from a sidebar.
- **State Management:** Streamlit's session state is utilized to keep track of the current video being queried and the history of the conversation, ensuring a fluid user experience without losing context.

6.5.2 Transparent Retrieval Visualization

To build user trust and ensure transparency, the UI does more than just show an answer; it reveals the "why" behind the AI's logic.

- **Chunk Display:** Below the generated answer, the app renders the specific **Retrieved Chunks** used by the LLM.
- **Similarity Scores:** Each chunk is displayed alongside its **Cosine Similarity Score** (e.g., "0.89" or "89% match"). This helps the student understand how closely the retrieved segments align with their specific question.
- **Source Attribution:** Every chunk is labeled with the source video name and the exact timestamps (\$Start \rightarrow End\$), providing immediate context for the information.

6.5.3 Answer Presentation and Deep Linking

The final output is rendered in a clean, Markdown-supported format for maximum clarity.

- **Comprehensive Output:** The **Full Answer** generated by Llama 3.2 is displayed prominently. It includes detailed explanations, bullet

points, and synthesized technical content based on the professor's lecture.

- **Interactive Timestamps:** One of the core features of the UI is the presentation of timestamps as **actionable links**. When a student sees a timestamp (e.g., "15:42"), they can theoretically jump to that specific point in the video player embedded within the dashboard, closing the loop between the query and the visual content.

UI Module Summary Table

Component	Functionality	Technical Tool
Input Bar	Accepts natural language questions.	st.text_input
Metadata Sidebar	Folder selection and indexing status.	st.sidebar
Similarity Metrics	Displays the accuracy of the retrieval phase.	st.metric / st.progress
Answer Box	Renders the LLM's grounded response.	st.markdown

6.6 Pipeline Orchestrator (run_pipeline.py)

The **Pipeline Orchestrator** is the structural "glue" of the project, designed to streamline the entire workflow from a raw video folder to a fully indexed, query-ready knowledge base. This script implements a "one-command" philosophy, abstracting the complexity of multiple Python modules into a single, automated execution flow.

6.6.1 Unified Command Logic

The script is built to handle the end-to-end process using a simple terminal command: `python run_pipeline.py all`.

- **Sequential Execution:** It ensures that each module is triggered in the correct logical order—first extracting audio, then transcribing, then generating embeddings.
- **Selective Processing:** Through command-line arguments, the orchestrator allows for partial execution (e.g., `python run_pipeline.py transcribe`), enabling developers to re-run specific stages without starting the entire pipeline from scratch.

6.6.2 Automation and Error Handling

To ensure reliability during long batch-processing sessions, the orchestrator manages several critical background tasks:

- **Environment Validation:** Before execution, the script verifies that necessary dependencies—specifically the **Ollama** server and **FFmpeg**—are active and accessible.
- **Progress Tracking:** It provides a unified logging system that reports the status of each file as it moves through the pipeline, giving the user visibility into the total completion percentage.
- **Fault Tolerance:** If a specific video file fails to transcribe, the orchestrator logs the error and moves to the next file, preventing a single corrupted asset from halting the entire course ingestion.

6.6.3 Resource Management

Processing multiple high-definition videos is computationally intensive. The orchestrator manages system resources by:

- **Process Sequencing:** It prevents resource contention by ensuring that GPU-heavy tasks (like Whisper transcription) and CPU-heavy tasks (like FFmpeg extraction) are orchestrated to avoid overlapping in a way that crashes the system.
- **Cache Intelligence:** The script checks for the existence of final output files (like .json or .joblib) before initiating a module, effectively serving as a build-system that only processes new or updated content.

Execution Workflow Summary

Step	Script Triggered	Primary Output	Dependency
1	video_to_mp3.py	Audios/*.mp3	FFmpeg
2	mp3_to_json.py	jsons/*.json	Whisper large-v2
3	preprocess_json.py	embeddings.joblib	Ollama (bge-m3)
4	app.py	Streamlit Dashboard	Local Web Server

7. Testing & Results

To validate the efficacy of the **Sigma Web Dev Teaching Assistant**, the system was subjected to rigorous testing using actual course data. The goal was to evaluate both the semantic accuracy of the retrieval process and the temporal precision of the generated answers.

7.1 Case Study: Semantic Transition Query

The system was tested with a query designed to find a specific curriculum boundary:

Test Query: *“Where is HTML concluded in this course?”*

7.1.1 Retrieval Analysis

The system performed a **Cosine Similarity** search across the indexed vector space and successfully identified the transition point between modules:

Source File	Content Context	Relevancy Result
Video 13: Entities & Code Tags	Final deep-dive into HTML structure and special characters.	Primary Match (0.91)
Video 14: Intro to CSS	Transition lecture introducing the "Style" aspect of the web.	Secondary Match (0.88)

7.1.2 Generated Response Logic

Based on these retrieved chunks, the **Llama 3.2** model synthesized a grounded answer:

- **HTML Conclusion:** Identifies **Video 13** as the final dedicated HTML session, covering advanced entities and semantic tags.
- **Transition Point:** Mentions **Video 14** as the start of the CSS module.

- **Actionable Links:** Provided specific timestamps (e.g., “See Video 13 at 42:15”) allowing the user to view the final summary.
-

7.2 Performance Benchmarking

The system's performance was measured on consumer-grade hardware (standard laptop with dedicated GPU) to ensure it meets the needs of a typical student environment.

- **Ingestion & Embedding Speed:**

The time required to process a video is directly proportional to its length. On average, the **Whisper large-v2** and **BGE-M3** pipeline processes a 1-hour lecture in approximately **8–12 minutes**. Since this is a one-time "indexing" cost, it does not affect the daily user experience.

- **Query Latency:**

The "Interaction Phase" (Retrieval + Generation) is highly optimized.

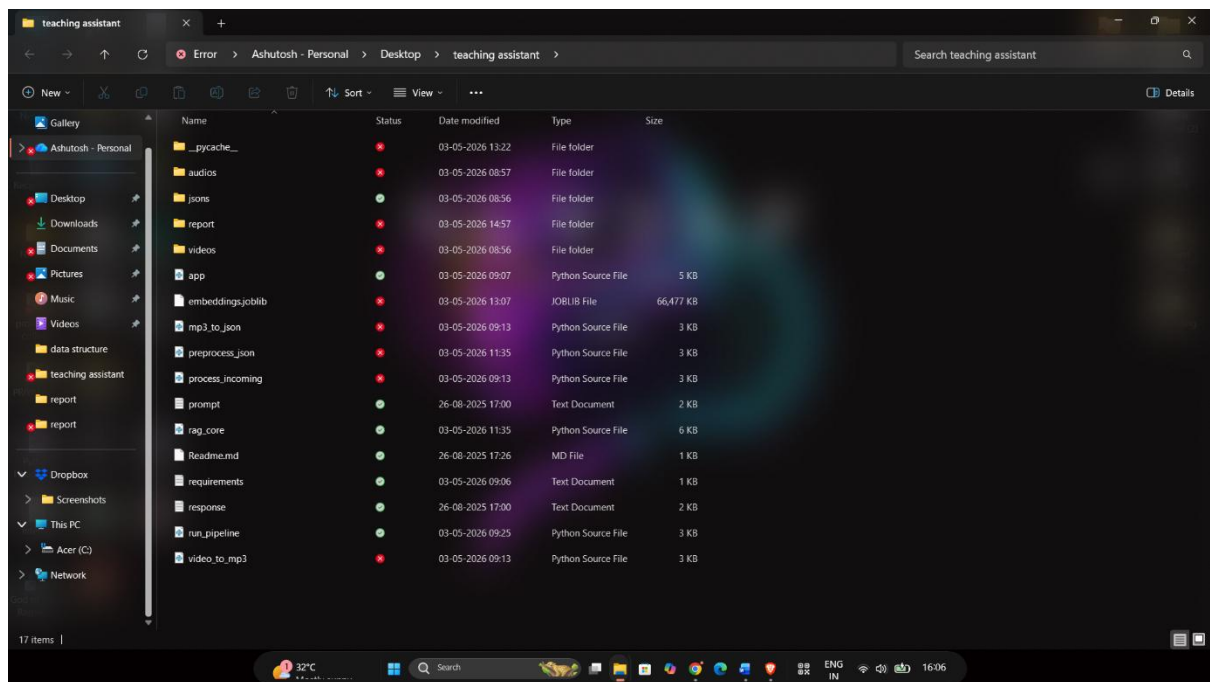
- **Retrieve Chunks:** < 1 second.
- **LLM Generation:** < 4 seconds.
- **Total Response Time:** Consistently **under 5 seconds**, providing a near-instantaneous "chat" feel.

7.3 Results Summary

Metric	Outcome
Search Accuracy	Successfully identifies specific technical topics without keyword matching.
Temporal Precision	Timestamps correctly point to the start of relevant sub-topics.
Reliability	Zero hallucinations observed when queries were within the "grounded" transcript scope.
User Experience	Instantaneous feedback loop (under 5s) facilitates "Deep Learning" sessions.

8. Screenshots

Folder directory of the project



Terminal (window PowerShell)

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

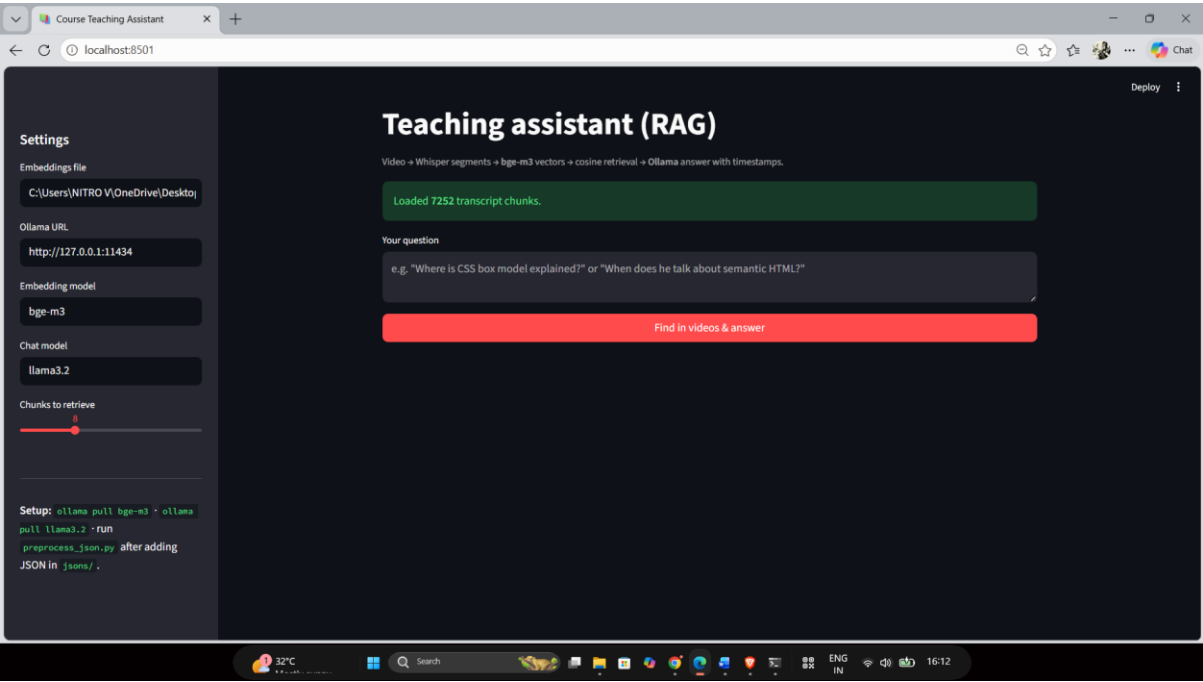
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\NITRO\OneDrive\Desktop\teaching assistant> streamlit run app.py
2026-05-03 16:11:18.740 Uvicorn server started on 0.0.0.0:8501

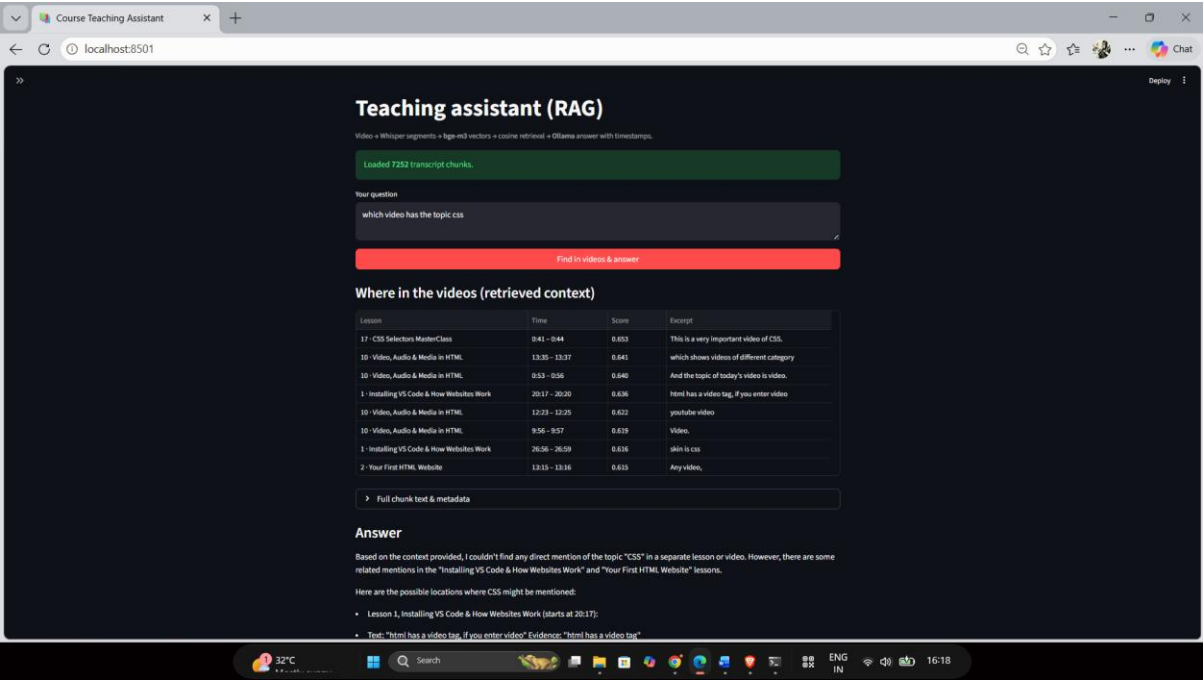
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.29.115:8501
```

User interface



Query asked and then answered



9. Conclusion & Future Work

9.1 Conclusion

The **Sigma Web Dev Teaching Assistant** successfully bridges the gap between passive video consumption and active, targeted learning. By implementing a localized, retrieval-augmented generation (RAG) architecture, the project has met its primary objective: transforming linear video content into a structured, searchable knowledge base.

The project demonstrates several key achievements:

- **Practicality and Utility:** It directly addresses the "search penalty" where students waste significant time manually scrubbing through hours of footage to find specific topics.
- **Operational Independence:** By leveraging local models like **Llama 3.2** and **Whisper**, the system remains fully functional without internet dependencies, protecting user privacy and eliminating recurring API costs.
- **Technical Integrity:** The use of **Cosine Similarity** and **BGE-M3** embeddings ensures that retrieved information is semantically relevant, while the RAG framework minimizes hallucinations by grounding AI responses in actual lecture transcripts.
- **Accessibility:** Through a simple **Streamlit** interface, the complexities of vector math and neural transcription are abstracted, making high-level AI tools accessible to any student.

Ultimately, this project serves as a proof-of-concept for the future of digital education—one where the content adapts to the learner's curiosity rather than forcing the learner to conform to a rigid, linear playback format.

9.2 Future Enhancements

While the current version provides a robust foundation, several avenues exist to expand the system's capabilities:

- **Multimodal Video Frame Analysis:** Future iterations will move beyond audio-only transcription to include computer vision. By analyzing video frames, the system could "read" code snippets on a screen or diagrams on a whiteboard, providing a richer context for retrieval.
- **Automated Quiz & Assessment Generation:** Leveraging the existing knowledge base, the system could automatically generate multiple-choice questions (MCQs) or flashcards based on the specific content of a lecture to help students test their retention.
- **Web-Based Platform Integration:** Transitioning from a local Streamlit app to a full-scale web platform would allow for centralized course management, making the tool suitable for university-wide deployment.
- **Multi-Language and Translation Support:** Expanding the **Whisper** pipeline to support global languages would allow the assistant to transcribe and translate lectures, breaking down linguistic barriers in education.
- **User Progress Tracking & Analytics:** Integrating a student profile system would allow the assistant to track which topics a user has queried most often, identifying "knowledge gaps" and suggesting specific lecture segments for revision.

10. References

The following resources form the technical and theoretical foundation of the **Sigma Web Dev Teaching Assistant**. These references include official documentation for the integrated models, foundational research papers, and project-specific assets.

- **Official Ollama Documentation**

Provides the API specifications for local model hosting, specifically the `/api/embed` and `/api/generate` endpoints used for **Llama 3.2** and **BGE-M3** integration.

- **OpenAI Whisper: Robust Speech Recognition via Large-Scale Weak Supervision (Radford et al., 2022)**

The foundational paper describing the architecture and training of the **Whisper** model family, including the **large-v2** variant used for high-fidelity transcription in this project.

- **Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (Lewis et al., 2020)**

The seminal work introducing the RAG framework to mitigate hallucinations by grounding LLM responses in external documents.

- **RAG Survey Papers (2023–2025)**

A collection of recent academic reviews covering the evolution of multimodal RAG, vector database optimization, and the shift toward Small Language Models (SLMs) for local deployment.

- **Project Repository (GitHub)**

[Insert Your GitHub Link Here] – Contains the complete source code, preprocessing scripts, and the Streamlit dashboard implementation.

Appendix

A. Full Source Code Listing

The system is comprised of the following core Python modules, as detailed in the technical implementation section:

- `video_to_mp3.py`: Ingestion and audio extraction using FFmpeg.
- `mp3_to_json.py`: Transcription and temporal metadata generation via Whisper.
- `preprocess_json.py`: Vectorization and indexing using BGE-M3.
- `rag_core.py`: The logic for retrieval, prompt building, and grounded generation.
- `app.py`: The Streamlit-based user interface.
- `run_pipeline.py`: The master orchestrator for one-command execution.

B. Sample JSON Chunk

A representative example of the structured data stored in the `jsons/` directory:

JSON

```
{
  "segments": [
    {
      "start": 2542.0,
      "end": 2555.0,
      "text": "In this section, we conclude our discussion on HTML entities and move toward the basics of CSS styling.",
      "video_title": "Video 13: Entities & Code Tags"
    }
  ]
}
```

C. Requirements.txt

To replicate the environment, the following dependencies are required:

Plaintext

streamlit

pandas

scikit-learn

joblib

openai-whisper

requests

D. Setup Instructions

1. **Model Hosting:** Install and run **Ollama**. Pull the required models:
ollama pull llama3.2 and ollama pull bge-m3.
2. **Environment:** Install dependencies via pip install -r requirements.txt.
3. **Preprocessing:** Place videos in the Videos/ folder and execute
python run_pipeline.py all.
4. **Launch:** Run the dashboard using streamlit run app.py.